

Database – Evaluating Schema Quality

Guidelines normalization

Database design – top-down

- Map conceptual model to relational database schemas
- Validate the schemas using guidelines and normalization
 - Check the quality of the schemas
- Importance of Domain Model
 - If the conceptual model is good, then there is less need for normalization.

Database design – top-down

- The implicit goal of the design activity:
- Information preservation
 - The relational design must preserve all concepts, which are originally captured in the conceptual design.
- Reduce redundancy
 - Avoid anomalies

Evaluating scheme quality

- Informal design guidelines
- Normalization

Informal guidelines

1. Make sure that the meaning of a table is clear.
 - Information about different things should not be placed in the same table.
2. Reduce redundant information in tuples
 - lead to update anomalies , i.e. problems at the insertion, modification and deletion of data
 - waste of storage
3. Reduce NULL values
 - Unexpected join results
 - Multiple interpretations
 - Waste of storage
4. Avoid generation spurious data during JOIN

1. Clear semantics

- Consider the following table for a library database :

Lending={title, copyNr, borrowerNr, borrowerName, borrowerStatus, date, reminderStatus}

- Information about different things are mixed in the same table

2. Reduce redundant values

- Anomaly:
 - something that leads to unexpected and/or unpredictable results.
- Types of anomalies:
 - Insert: when we only want to insert part of the data / when we need to match data already inserted
 - Delete: when a value we want to delete also means we will delete values we wish to keep.
 - update: when we want to change a single data item value, but must update multiple entries
- Need to check your database design carefully:
 - the database must be anomaly free.

Example Redundancy and Anomalies

(1) let's say, we have this bad design (Employee and Department in the same table)

EMP_DEPT						
Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn

Cf. [Elmasri] fig. 14.11 (b)

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
James Borg	888665555	1937-11-10 00:00:00.000	450 Stone, Houston, TX	1	Headquarters	888665555
Jennifer S Wallace	987654321	1941-06-20 00:00:00.000	291, Berry, Belaire, TX	4	Administration	987654321
Ahmad V Jabbar	987987987	1969-03-29 00:00:00.000	980 Dallas Houston, TX	4	Administration	987654321
Alicia J Zelaya	999887777	1968-01-19 00:00:00.000	3321 Castle, Spring, TC	4	Administration	987654321

Insert Anomalies

When inserting an Employee, we need to specify values for department as well for each Employee and make sure they are consistent between rows (not good)

How do we insert just a new department?

Then we have to insert null for Employee attributes -> but we can't do that for the primary key Ssn

(2) then we end up with duplicate values for Dnumber, Dname and Dmgr_ssn

Example Redundancy and Anomalies

(1) let's say, we have this bad design (Employee and Department in the same table)

EMP_DEPT						
Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn

Cf. [Elmasri] fig. 14.11 (b)

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
James Borg	888665555	1937-11-10 00:00:00.000	450 Stone, Houston, TX	1	Headquarters	888665555
Jennifer S Wallace	987654321	1941-06-20 00:00:00.000	291, Berry, Belaire, TX	4	Administration	987654321
Ahmad V Jabbar	987987987	1969-03-29 00:00:00.000	980 Dallas Houston, TX	4	Administration	987654321
Alicia J Zelaya	999887777	1968-01-19 00:00:00.000	3321 Castle, Spring, TC	4	Administration	987654321

Deletion Anomalies

If we delete an Employee that happens to represent the last employee working for a particular department, the information concerning that department is lost from the database

(2) then we end up with duplicate values for Dnumber, Dname and Dmgr_ssn

Example Redundancy and Anomalies

(1) let's say, we have this bad design (Employee and Department in the same table)

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------

Cf. [Elmasri] fig. 14.11 (b)

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
James Borg	888665555	1937-11-10 00:00:00.000	450 Stone, Houston, TX	1	Headquarters	888665555
Jennifer S Wallace	987654321	1941-06-20 00:00:00.000	291, Berry, Belaire, TX	4	Administration	987654321
Ahmad V Jabbar	987987987	1969-03-29 00:00:00.000	980 Dallas Houston, TX	4	Administration	987654321
Alicia J Zelaya	999887777	1968-01-19 00:00:00.000	3321 Castle, Spring, TC	4	Administration	987654321

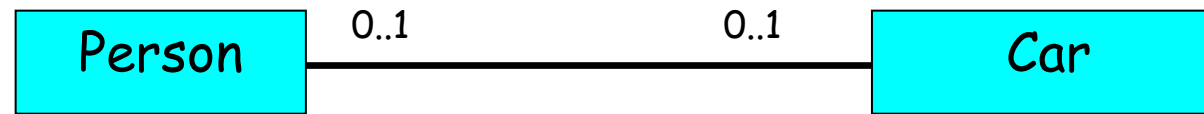
Modification Anomalies

If we change the value of one of the attributes of a particular department - say, the manager of department 5 - we must update the tuples of all employees who work in that department; otherwise, The database will become inconsistent.

(2) then we end up with duplicate values for Dnumber, Dname and Dmgr_ssn

3. Reduce NULL values

- If NULL is the most common value for an attribute, then that attribute may not belong in the table.



- One out of ten employees has a company car.
- One out of ten cars are *not* assigned to a certain employee.
- On which side should the foreign key be included?

4. Spurious tuples

- Consider the following sample of an "alternative" table for a library database:
 - Borrower = {BNR, fname, **Iname**,}
 - Copy = {copyNr, ..., **Iname** ...}
- The association Loan between Borrower and Copy is designed by having the borrower's last name included in Copy.
- When we join Borrower and Copy, spurious tuples will occur since Borrower.Iname is hardly unique.
- That means that borrower '117 Ib Nielsen' will act as a borrower of all copies where Iname = 'Nielsen'- i.e. of all copies borrowed by all the other with last name 'Nielsen' (which will also act as borrower of Ib's copies).

Decomposition

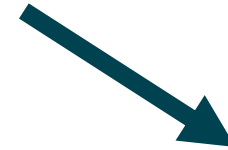
- Decomposition is the process of breaking down in parts or elements.
 - It replaces a relation with a collection of smaller relations.
 - It breaks the table into multiple tables in a database.
 - It should always be *lossless, because it confirms that the information in the original relation can be accurately reconstructed based on the decomposed relations.
 - If there is no proper decomposition of the relation, then it may lead to problems like loss of information.
-
- *Lossless means that the information should not get lost from the relation that is decomposed. It gives a guarantee that the join will result in the same relation as it was decomposed.

Decomposition

Eid	Ename	Age	City	Salary	DeptId	DeptName
1	Torben	45	Brønderslev	30000	1	Finance
2	Istvan	32	Frederikshavn	35000	2	Sales
3	Anita	37	Aalborg	32000	3	Marketing
4	Bodil	30	Aalborg	37000	4	Development
5	Henrik	29	Brønderslev	29000	5	Human Ressource



Eid	Ename	Age	City	Salary
1	Torben	45	Brønderslev	30000
2	Istvan	32	Frederikshavn	35000
3	Anita	37	Aalborg	32000
4	Bodil	30	Aalborg	37000
5	Henrik	29	Brønderslev	29000



Deptid	Eid	DeptName
1	1	Finance
2	2	Sales
3	3	Marketing
4	4	Development
5	5	Human Ressource

Normalization

- Formally measure why one set of groupings of attributes into relation schemas is better than another
- Normal forms (NF) provide formal guideline for the design of the tables.
- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - the decomposition is a *lossless-join decomposition
 - each relation is in good form
- *lossless-join decomposition means that natural joins of all the decompositions give the original relation.

Normalization

- We look at:
 - 1NF, 2NF, 3NF and Boyce-Codd Normal Form (BCNF)
 - Based on functional dependencies
- Not part of the material covered
 - 4NF could have been considered
 - 5NF and 6NF
- The ideal normal form:
 - No anomalies, no lost information
 - 3NF and BCNF are the most interesting in practice
 - 3NF and BCNF eliminate redundancy

First normal form 1NF

- *“First normal form (1NF) is now considered to be part of the formal definition of a relation in the basic (flat) relational model; historically, it is defined to disallow multivalued attributes, composite attributes, and their combinations.”*
 - P. 507 in *Elmasri*
- Only atomic values are allowed

It follows from the relational model and 1NF...

1. The order of the rows and columns does not matter
2. Rows must be unique (Must have PK)
3. Each column must contain a single value
4. Columns cannot contain repeated groups

First normal form 1NF

- A relation that is not in 1NF

DEPARTMENT			
DNAME	<u>DNUMBER</u>	DMGRSSN	DLOCATIONS
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

- Remedy:
 - Form new relations for each multivalued attribute or nested relation

Exercise

- Solve exercise 1

Definition of Functional Dependency (FD)

- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines the value for Y .
- Any tuples which agree in (have the same value for) all attributes of X must also agree in all attributes of Y
- Written as $X \rightarrow Y$;
- X is *determinant* set; Y is the *dependent* set.
- Specifies a constraint on all relation instances $r(R)$

Functional Dependency (FD)

Examples:

$Ssn \rightarrow Ename$

The value of the employee's Social Security number (Ssn) uniquely determines the employee's name

$Pnumber \rightarrow \{Pname, Plocation\}$

The value of a project's number (Pnumber) uniquely determines the project name and location

Examples of FD constraints

<u>Employee</u>	Salary	<u>Project</u>	Balance	Role
-----------------	--------	----------------	---------	------

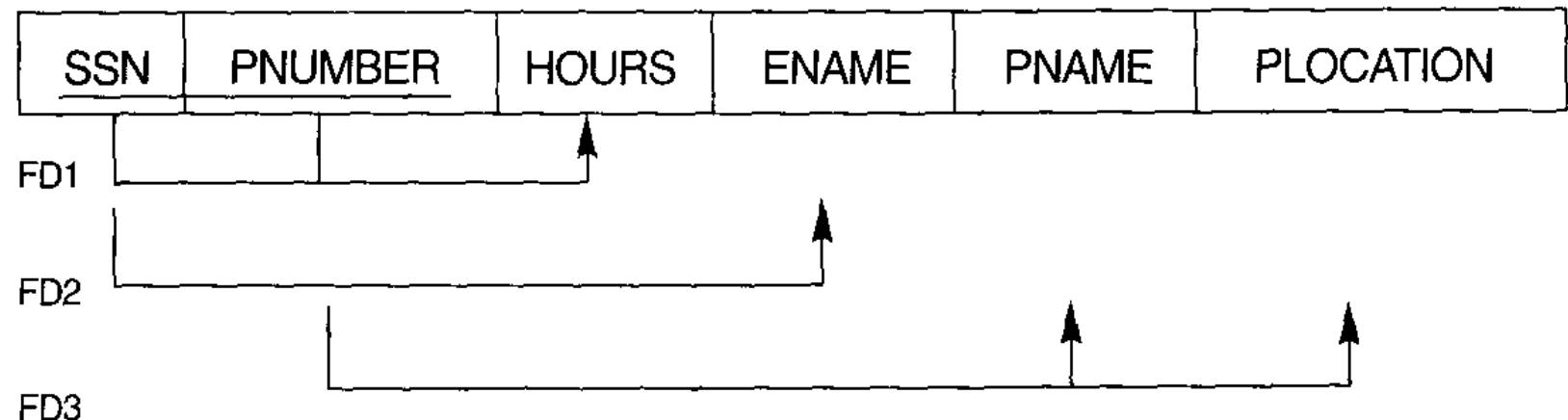
- Example of FDs
 - Employee → Salary
 - Project → Balance
 - Employee, Project → Role
- Trivial FD
 - Employee, Project → Project

Employee	Salary
4711	25000
..	..
4711	25000
..	..
4712	20000
4713	25000

Examples of FD constraints

- FD1: {SSN, PNUMBER} → HOURS
 - *Ssn and project number determines hours*
- FD2: SSN → ENAME
 - *Social security number determines employee name*
- FD3: PNUMBER → {PNAME, PLOCATION}
 - *Project number determines project name and location*

EMP_PROJ



Identifying FD

- FDs are derived from the real-world constraints on the attributes
- FDs are domain knowledge
 - Intrinsic features of the data you are dealing with
 - Something you know (or assume) about the data
- Look to the table: coincidence or FD?
Color → Area
Domain knowledge is required to validate FDs

Color	Area
Blue	27
Green	11
Blue	27

Second Normal Form 2NF

1. It is in 1NF
 2. All of the non-key fields depend on all of the key fields
- **Relevant only if the key consists of more than 1 attribute**
(composite key)
 - All attributes in the relations must be functional dependent of **the whole key** – and not only by part of the key

Second Normal Form 2NF

- A relation schema R is in second normal form (2NF) if every non-prime attribute A in R is fully functionally dependent on the candidate key.
- In other words, if non-prime attribute A is not dependent on any proper subset of any candidate key.
- Ok?

<u>cpr</u>	<u>zip</u>	foreName	surName	zipCode_city
------------	------------	----------	---------	--------------

Normalize to 2NF

- Decompose and set up a new relation for each partial key with its dependent attribute(s).
- *Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.*
- *Decompose:*

<u>cpr</u>	<u>zip</u>	foreName	surName	zipCode_city
------------	------------	----------	---------	--------------

Second Normal Form 2NF

- Another counterexample : not in 2NF
 - Movies(title, year, star, studioName, studioAddress, salary)
 - FD: {title, year} → {studioName, studioAddress}

<u>Title</u>	<u>Year</u>	<u>Star</u>	StudioName	StudioAddr	Salary
Star Wars	1977	Hamill	Lucasfilm	1 Lucas Way	\$100M
Star Wars	1977	Ford	Lucasfilm	1 Lucas Way	\$100M
Star Wars	1977	Fisher	Lucasfilm	1 Lucas Way	\$100M
Patriot Games	1992	Ford	Paramount	Cloud 9	\$2000M
Last Crusade	1989	Ford	Lucasfilm	1 Lucas Way	\$1000M

Normalize to 2NF

- Decompose and set up a new relation for each partial key with its dependent attribute(s).
- Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.

Movies (title, year, studioName, studioAddress)

Actors (title, year, star, salary)

Exercise

- Solve exercise 2

Third normal form 3NF

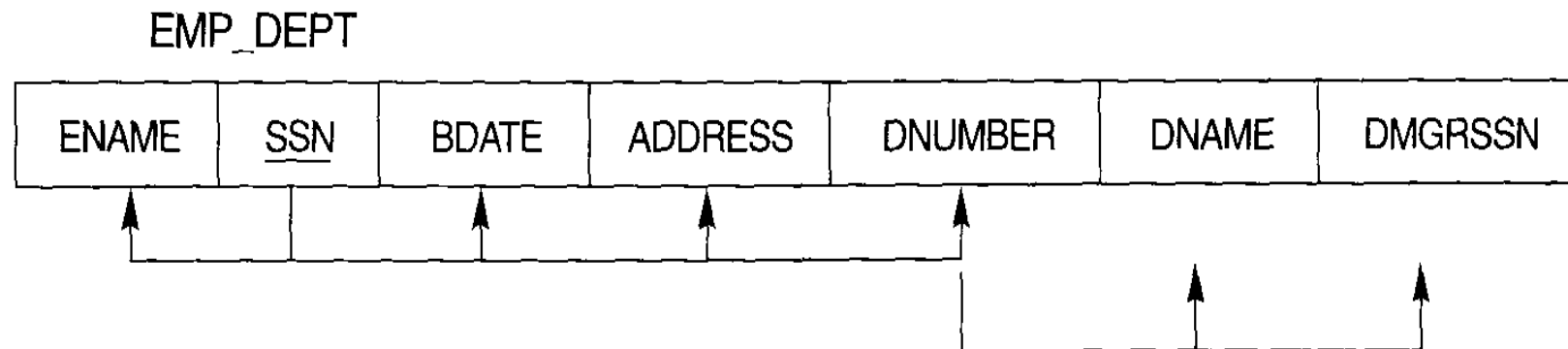
1. It is in 2NF
 2. It contains no transitive dependencies
- A transitive dependency is *when one non-key field's value depends on another non-key field's value.*
 - Any nonprime attribute must not determine another nonprime attribute.
 - ?

<u>cpr</u>	zip	foreName	surName	zipCode_city
------------	-----	----------	---------	--------------

Third normal form 3NF

Examples:

- $SSN \rightarrow DMGRSSN$ is a *transitive* FD since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ hold
- $SSN \rightarrow ENAME$ is *non-transitive* since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow ENAME$



Third normal form 3NF

- Counterexample : not in 3NF
 - Job(empname, city, country)
 - FD: city → country

Light blue means key

empname	city	country
joe	chicago	usa
sam	chicago	usa
ted	chicago	usa
job	chicago	usa
brian	newyork	usa
mary	newyork	usa
anil	newyork	usa

- Redundant that Chicago is in USA

Third normal form 3NF

- Counterexample : *now* in 3NF
 - Job (empname, city)
 - CityLocation (city, country)

empname	city
joe	chicago
sam	chicago
ted	chicago
job	chicago
brian	newyork
mary	newyork
anil	newyork

city	country
chicago	usa
newyork	usa

Normalize to 3NF

- Decompose and set up a relation that includes the *nonprime attribute(s)* that functionally determine(s) other nonprime attribute(s).

Exercise

- Solve exercise 3

Boyce-Codd Normal Form (BCNF)

- **Boyce–Codd normal form** (or **BCNF** or **3.5NF**) is a normal form used in database normalization. It is a slightly stronger version of the third normal form (3NF). BCNF was developed in 1974 by Raymond F. Boyce and Edgar F. Codd to address certain types of anomalies not dealt with by 3NF as originally defined.



Boyce-Codd Normal Form (BCNF)

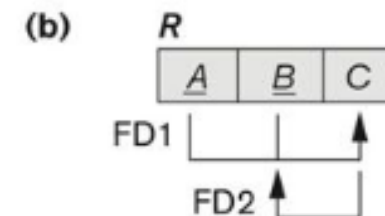
- Boyce-Codd Normal Form (BCNF)
 - A relation is said to be in BCNF if
 - The relation is in Third Normal Form (3NF)
 - And every determinant is a candidate key
- For each $A \rightarrow B$, A is a key
 - For all functional dependencies, it must be the case that the determinant is a candidate key.
- The data you store in a table must **depend on the key, the whole key and nothing but the key**
 - “So help me Codd”

*Determinant!
Candidate key!*

Boyce-Codd Normal Form (BCNF)

- Not in BCNF general example
- $R(A,B,C)$, $C \rightarrow A$ where:
 - The relation contains attributes A, B and C
 - A and B form a candidate key
 - C is not part of a candidate key
 - C functionally determines B
 - B is functionally dependent on C

Elmasri, fig. 14.13, p. 518



Nearest shops – 3NF but not BCNF

<u>Person</u>	<u>ShopType</u>	<u>NearestShop</u>
Pedersen	Optician	ProfilOptik
Pedersen	Hairdresser	SalonHair
Nielsen	Bookshop	BogHandel
Jensen	Bakery	GuldBager
Jensen	Hairdresser	Klipotetek
Jensen	Optician	ProfilOptik



<u>NearestShop</u>	<u>ShopType</u>
ProfilOptik	Optician
SalonHair	Hairdresser
BogHandel	Bookshop
GuldBager	Bakery
Klipotetek	Hairdresser

- The candidate keys of the table are:
 - {Person, Shop Type}
 - {Person, Nearest Shop}
- The FDs are:
 - Person, ShopType → NearestShop
 - NearestShop → ShopType

<u>NearestShop</u>	<u>Person</u>
ProfilOptik	Pedersen
SalonHair	Pedersen
BogHandel	Nielsen
GuldBager	Jensen
Klipotetek	Jensen
ProfilOptik	Jensen

Boyce-Codd Normal Form (BCNF)

Take the following table structure as an example:

- schedule(campus, course, class, time, building)
- Take the following sample data:

campus	course	class	time	building
East	English 101	1	8:00-9:00	212 AYE
East	English 101	2	10:00-11:00	305 RFK
West	English 101	3	8:00-9:00	102 PPR

Boyce-Codd Normal Form (BCNF)

Note that no two buildings on any of the university campuses have the same name, thus **BUILDING** $\rightarrow$ **CAMPUS**. As the determinant is not a candidate key this table is **NOT** in Boyce-Codd normal form.

campus	course	class	time	building
East	English 101	1	8:00-9:00	212 AYE
East	English 101	2	10:00-11:00	305 RFK
West	English 101	3	8:00-9:00	102 PPR

BCNF vs 3NF

- BCNF
 - Stricter than 3NF
- 3NF
 - Always allows lossless-join and dependency preserving
 - May allow some anomalies
- “In practice, most relation schemas that are in 3NF are also in BCNF”

Is normalization a must?

- If the domain model is well thought out, then the schema derived from transformation rules are normally good.
 - *Still check it.*
- If a relation is not in the desired normal form
 - then decompose it
- The DB must be on BCNF

Exercises

- Solve exercises 4 and 5